

MLBDA – MU4IN801-2^E EXAMEN REPARTI

DU 13 JANVIER 2021

Ex 1 :	Ex 2 :	Ex 3 :	
--------	--------	--------	--

Les documents sont autorisés – Durée totale : 1h30.

Répondre aux questions sur la feuille du sujet dans les cadres appropriés. La qualité de la rédaction sera prise en compte dans la note.

Exercice 1. SQL3

4 pts

Le schéma suivant permet de stocker les lignes de transports (metro, bus, tram, RER) avec leurs arrêts et les connexions :

```
create type T_Arret as object (  
  nom_arret varchar(64),  
  position number(7)  
) final instantiable;  
/  
create type Ens_Arrets as table of T_Arret;  
/  
create type T_Ligne as object (  
  nom_ligne varchar(16),  
  type_ligne integer,
```

```
  arrêts Ens_Arrets  
) final instantiable;  
/  
create type T_Connexion as object (  
  ligne_1 ref T_Ligne,  
  arret_1 T_Arret,  
  ligne_2 ref T_Ligne,  
  arret_2 T_Arret  
) final instantiable;  
/
```

On connaît le nom, le type (tram=0, metro=1, RER=2, bus=3) et les arrêts de chaque ligne. Pour chaque arrêt on stocke le nom et sa position dans le parcours de la ligne. Une connexion entre deux lignes est représentée par un objet de type T_Connexion.

Question 1. Définissez les racines de persistance (tables) LesLignes et LesConnexions pour stocker les lignes et les connexions

```
create table of LesLignes of T_Lignes nested table arrêts store as tab1;
```

```
create table LesConnexion of T_connexion;
```

Question 2.

Insérez un arrêt à la station 'Jussieu' (nom_arret) dans la ligne de de métro 7 à la position 10 et dans la ligne de métro 10 à la position 2. On suppose que les deux lignes existent déjà dans la base.

Question 3. Insérez une connexion entre les deux arrêts de la ligne 7 et de la ligne 10 à la station Jussieu. Attention : on suppose que la table LesConnexions est symétrique.

```
INSERT INTO LesConnexion VALUES (  
  (SELECT REF(l) FROM LesLignes l WHERE l.nom_ligne = '10'), -- 线路 10 的引用  
  T_Arret('Jussieu', 2), -- 站点 'Jussieu' 在线路 10 上的实例  
  (SELECT REF(l) FROM LesLignes l WHERE l.nom_ligne = '7'), -- 线路 7 的引用  
  T_Arret('Jussieu', 10) -- 站点 'Jussieu' 在线路 7 上的实例  
);
```

Question 4. Ecrivez l'instruction SQL3 qui trouve tous les couples de lignes de métro qui se croisent au moins 3 fois (trois connexions à des stations différentes).

Exercice 2. XML**8 Pts**

On considère le fichier *Bibliothèque.xml* suivant :

<pre><?xml version="1.0" encoding="UTF-8"?> <bibliothèque> <ouvrage ed='E1' type='livre'> <description> <titre>HistoryOfFutureCities</titre> <auteur>Brook</auteur> Every month, five million people... </description> <pages nb=480/> </ouvrage> <ouvrage ed='E2' type='livre'> <description> <titre>L'ombre du vent</titre> <auteur>Zafon</auteur> 1945.Barcelone se réveille... </description> <pages nb=505/> </ouvrage> <ouvrage ed='E2' type='biographie'> <description> <titre>Mémoires de mon père</titre> <auteur>Olivier IKOR</auteur> </description> <pages/> </ouvrage> </bibliothèque></pre>	<pre><ouvrage ed='E1' type='livre'> <description> <titre>The Perfect Storm</titre> <auteur>Junger</auteur> It was the storm of the century... </description> <pages/> </ouvrage> <ouvrage ed='E2' type='biographie'> <description> <titre>Prisonnier du ciel</titre> <auteur>Zafon</auteur> </description> <pages nb=352> </ouvrage> <editeur n='E2' nom='Laffont'> <pays>France</pays> </editeur> <editeur n='E1' /> </bibliothèque></pre>
---	--

Question 1 (1pt). Pour chaque déclaration suivante de l'élément bibliothèque, le document *Bibliothèque.xml* précédent est-il conforme ? (**justifiez**) :

- a) < !ELEMENT bibliothèque (editeur*, ouvrage*, (ouvrage | editeur ?))>
- b) < !ELEMENT ouvrages (editeur ?, ouvrage+, ouvrage*, editeur+)>
- c) < !ELEMENT ouvrages (editeur ?, (ouvrage+ | editeur+), ouvrage*)>
- d) < !ELEMENT ouvrages (((ouvrage | editeur)* | editeur *), ouvrage ?, editeur*)>

Déclaration(s) compatible(s) avec le document XML :

Justification :

XPATH

Question 2 (1pt). Donnez le nombre d'éléments renvoyés par chaque expression XPATH suivante :

a) `//*[ @ed='E2' ]`

b) `/bibliothèque/*[pages]//titre`

c) `//*[ @*='E1' ]`

a) 3

b) 3

c) 0

Question 3 (3pts). Exprimez en XPath les requêtes suivantes sur le fichier *Bibliothèque.xml* :

a) Le pays des éditeurs des ouvrages qui ont plus de 350 pages.

```
/bibliothèque/éditeur[@n=/bibliothèque/ouvrage[pages/@nb > 350]/@ed]/pays
```

b) Le nom des éditeurs dont le pays est inconnu.

```
/bibliothèque/éditeur[not(pays)]/@nom
```

c) Le troisième livre du document.

```
/bibliothèque/ouvrage[3]/description/titre
```

XQuery

Exprimez en XQuery les requêtes suivantes :

Question 4 (2pts). Les éditeurs dont tous les livres édités ont un nombre de pages connu.

Le résultat doit satisfaire la DTD suivante :

< !ELEMENT éditeur EMPTY>

< !ATTLIST éditeur id ID>

```
let $ouvrages := /bibliothèque/ouvrage
for $editeur in /bibliothèque/editeur
where every $ouvrage in $ouvrages[@ed = $editeur/@n] satisfies exists
($ouvrage/pages/@nb)
return <éditeur id="{ $editeur/@n }"/>
```

Question 5 (1 pt). Livres pour lesquels il existe au moins une biographie éditée par le même éditeur. Le résultat doit satisfaire la DTD suivante :

< !ELEMENT livre EMPTY>

< !ATTLIST livre titre CDATA>

```
let $biographies := /bibliothèque/ouvrage[@type = 'biographie']
for $livre in /bibliothèque/ouvrage[@type = 'livre']
where some $bio in $biographies satisfies $bio/@ed = $livre/@ed
return <livre titre="{ $livre/description/titre }"/>
```

Exercice 3. SPARQL

8 pts

Considérons les triplets RDF sous format Turtle qui décrivent les pays, les frontières entre pays, ainsi que quelques rivières qui les traversent.

Pour chaque pays (sujet de type Country ayant le préfixe count et qui sert d'identifiant) on connaît son nom (propriété name), sa population (propriété du même nom) et les organisations auxquelles il appartient (propriété isMember). Pour chaque frontière (sujet de type Border) on connaît sa longueur (propriété length) et les pays qu'elle sépare (propriété isBorderOf). Pour chaque rivière (sujet de type River) on connaît sa longueur (propriété length) et les pays qu'elle traverse (propriété isLocatedIn).

Ces données sont données à titre indicatif.

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix : <http://www.semwebtech.org/mondial/10/meta#> .
@prefix count: <http://www.semwebtech.org/mondial/10/countries/> .
@prefix org: <http://www.semwebtech.org/mondial/10/organizations/>.
@prefix r: <http://www.semwebtech.org/mondial/10/rivers/>.

count:F a :Country ;
    :name "France" ;
    :population 65 ;
    :isMember org:UN , org:NATO , org:OECD , org:UNESCO .
count:I a :Country ;
    :name "Italy" ;
    :population 59 ;
    :isMember org:UN , org:NATO , org:OECD .
count:D a :Country ;
    :name "Germany" ;
    :population 82 ;
    :isMember org:UN , org:NATO , org:OECD , org:ISO .
count:E a :Country ;
    :name "Spain" ;
    :population 42 ;
    :isMember org:UN , org:NATO , org:OECD , org:ISO .
count:R a :Country ;
    :name "Russia" ;
    :population 143 ;
    :isMember org:UN , org:ISO .
```

```

:fi a :Border ;    :length 450 ;    :isBorderOf count:F , count:I .
:fe a :Border ;    :length 620 ;    :isBorderOf count:F , count:E .
:fd a :Border ;    :length 480 ;    :isBorderOf count:F , count:D .
:db a :Border ;    :length 167 ;    :isBorderOf count:D , count:B .
:dp a :Border ;    :length 456 ;    :isBorderOf count:D , count:PL .
:ia a :Border ;    :length 430 ;    :isBorderOf count:I , count:A .
:ep a :Border ;    :length 1214 ;    :isBorderOf count:E , count:P .

:seine a :River ; :length 776; :locatedIn count:F .
:rhein a :River ; :length 1324; :locatedIn count:F , count:D, count:CH .
:saar a :River ; :length 227; :locatedIn count:F , count:D .

```

Question 1. Combien y a-t-il de sujets distincts et de propriétés distinctes dans les données ? (0.5 pt)

Nombre de sujets distincts :

Nombre de propriétés distinctes :

Exprimer en SPARQL les requêtes qui retournent les informations suivantes :

Question 2. Les noms des pays membres de l'organisation NATO et ayant une population supérieure à 50. (1 pt)

SELECT ?n

WHERE

{

.....

.....

.....

FILTER (.....)

}

Question 5. Les paires d'identifiants des pays qui ont exactement deux organisations en commun. Ne pas répéter les paires symétriques. (1.5 pt)

```

SELECT      .....      ?p1 ?p2

WHERE      SELECT ?country1 ?country2
           WHERE {
{           ?country1 a :Country .
           ?country2 a :Country .
           ..... ?country1 :isMember ?org1 .
           ..... ?country2 :isMember ?org1 .
           ..... ?country1 :isMember ?org2 .
           ..... ?country2 :isMember ?org2 .
           ..... FILTER (?country1 != ?country2 && ?org1 != ?org2)

           ..... # 确保不重复对称的国家对
           ..... FILTER (STR(?country1) < STR(?country2))

           ..... # 确保只有两个共同组织
           ..... FILTER NOT EXISTS {
           ..... ?country1 :isMember ?org3 .
           ..... ?country2 :isMember ?org3 .
           ..... FILTER (?org3 != ?org1 && ?org3 != ?org2)
           ..... }
           ..... }
           .....
           .....
           .....
}

```

Question 6. Les rivières traversant au moins deux pays, et pour celles qui en traverse un troisième retourner le nom de ce pays. (1.5 pt)

```

SELECT      ?r ?ntp

WHERE      SELECT ?river ?thirdCountry
           WHERE {
{           ?river a :River .
           ..... ?river :locatedIn ?country1 .
           ..... ?river :locatedIn ?country2 .
           ..... FILTER (?country1 != ?country2)

           ..... OPTIONAL {
           ..... ?river :locatedIn ?thirdCountry .
           ..... FILTER (?thirdCountry != ?country1 && ?thirdCountry != ?country2)
           ..... }
           ..... }
           .....
           .....
           .....
           Filter ( ..... )
           .....
}

```

Question 7. Les pays qui appartiennent à toutes les organisations auxquelles appartient l'Italie. (1.5 pt)

```
SELECT ?c          SELECT ?c
WHERE {             WHERE {
# 获取意大利所属的所有组织
# 获取属于这些组织的所有国家
# 确保这些国家属于意大利所在的所有组织
FILTER NOT EXISTS {
?c2 :isMember ?orgItaly .
FILTER NOT EXISTS {
?c :isMember ?orgItaly.
}
}
}
```